Mohammad Kashmar

Dashboard and calculations backend (~60%); dashboard/profile/duty UI (~40%)

Branch: Mohammad | Plan begins when the team receives the starter.

Own the shared stock calculations and pharmacist dashboard API, then build dashboard and pharmacy profile/duty screens. Agree response types early and create your own small local test fixtures to build screens before Hamid finishes his endpoints.

Your code ownership

- apps/api/src/modules/dashboard/**
- apps/api/src/common/inventory-calculations.ts and its tests
- apps/web/src/features/dashboard/**
- apps/web/src/features/pharmacy/**

Your agreed interfaces

- GET /pharmacist/dashboard -> Dashboard

- calculateMedicine(batches,threshold,today) and calculateDashboard(medicines,today): agree signatures with Mustafa Day 1.

- Dashboard frontend calls your API; profile/duty frontend calls Hamid's endpoints through feature api.ts.

- Backend calculates all indicators; no editable status fields and no authoritative frontend calculations.

Start without waiting

Pull the foundation from main into your branch. Run npm ci, copy both apps/api/.env.example and apps/web/.env.example to .env on first setup, then npm run dev. The frontend shows only a plain startup message. No database models/migrations/seed or product features are implemented. Mustafa authors the database setup; follow his instructions after that PR merges.

Read the final specification, proposed API contract, acceptance checklist and your task folders. You build the assigned pages/services yourself. There are no prebuilt guards, calculations, forms, components, adapters or fixtures. Agree interfaces early and create your own small local test data to work independently.

Scope boundary

Do not implement inventory mutations, authentication, owner services or public search. Do not store calculated statuses or recompute authoritative totals in React.

Week 1 - build your area

Days are relative, not the instructor deadline. Day 7 is a review/buffer day; adjust to your availability. Deliver small reviewed PRs throughout the week.

Day 1

Agree calculation helper input/output with Mustafa and publish the types. Read all indicator/dashboard rules and write a small hand-calculated fixture with expired, healthy, zero and low+expiring stock.

Day 2

Implement/test pure date/quantity/indicator helper in common/inventory-calculations.ts. Deliver this small PR first so Mustafa can use it without waiting for the dashboard.

Day 3

Test yesterday/today/+90/+91 expiry boundaries, available sums, quantity==threshold and simultaneous indicators. HEALTHY ignores empty expired/soon batches, but nonempty ones prevent healthy.

Day 4

Implement dashboard aggregation with Prisma.Decimal: nonexpired units/value, expiring batch count, low-stock medicines, expired batch count and positive expired waste value. Add hand-calculation tests.

Day 5

Implement own-pharmacy dashboard service/controller using shared auth/role/ownership foundations. Inactive subscription reads remain allowed. Add all attention items with multiple indicators.

Day 6

Document/test dashboard in Swagger. Give Mustafa fixed-date response fixtures and compare inventory filters with the same helper. Keep all calculations in backend services/pure functions.

Day 7

Buffer/review: publish your backend PR. Start dashboard cards and profile/duty forms from scratch with your own feature-local sample data; coordinate shared routing/styles with Alaa.

Week 2 - integrate and prove

Day 8: API integration. Day 12: core acceptance. Days 13-14: English rehearsal and fixes. No new optional features while CORE is failing.

Day 8

Finish backend regression tests: ownership, inactive reads, empty pharmacy, decimal costs and no expired units in available totals. Connect DashboardPage to your real endpoint.

Day 9

Build all required dashboard metrics, pharmacy name, subscription banner and attention list. Show all relevant badges and loading/empty/error states.

Day 10

Build pharmacy profile form with editable area/address/phone only. Show Hamid's backend validation messages and disable changes for inactive subscription while retaining read access.

Day 11

Build duty list/add/edit/delete forms. Convert datetime-local values with explicit local timezone to UTC ISO; show current periods clearly and surface overlap/end/duration errors.

Day 12

Integrate Hamid's real profile/duty APIs. Refresh views after mutations; check delete confirmation, successful form reset and retained values after errors. Coordinate dashboard refresh after Alaa's inventory changes.

Day 13

Use real API integration for all your screens. Reconcile hand-calculated totals with persisted stock and fix discrepancies with Mustafa. Rehearse keyboard/mobile/error cases.

Day 14

Buffer/final rehearsal. Explain one calculation in English, show multi-indicator attention and an invalid duty change. Know how your shared helper is used by inventory too.

Ready-to-merge checklist

[ ] A09: expiry boundaries and calendar days correct; no editable status data.

[ ] A10: LOW_STOCK at equality, multiple indicators retained and correct healthy/nonempty handling.

[ ] A11: 10x2.50 nonexpired +3x4.00 expired yields value 25.00/waste 12.00/units 10.

[ ] Dashboard scoped to own pharmacy and available to inactive subscribers; empty data has honest zero/empty values.

[ ] A16: all 3 frontend areas handle loading/empty/error, validation and inactive read-only state; profile/duty mutations refresh.

Handoffs and dependencies

- Critical: Day 1 signatures and Day 2 helper PR for Mustafa; this is the main backend dependency.

- Independent UI work: create local dashboard/profile/duty fixtures from agreed response types while Hamid builds his endpoints.

- Day 7: dashboard API available. Day 8: get Hamid's profile/duty endpoint examples.

- Day 12: coordinate a refresh mechanism with Alaa; avoid editing each other's pages directly.

Every pull request

Commit only your intended files; push your branch; open a PR into main. Include the task/rule IDs, a short explanation, and tests or screenshots. Run npm run typecheck, npm test and npm run build. Backend contributors also document their endpoints in Swagger. Alaa reviews/merges; keep .env, node_modules, dist and temporary files untracked.

Your English presentation segment

Explain a dashboard total from actual batches, show a medicine with two indicators and demonstrate a duty/profile edit. Describe why expired stock still contributes to waste but not availability.

Everyone should understand the whole flow: owner onboarding -> pharmacist stock -> dashboard -> public search. Be ready to explain a rule outside your own feature. Final acceptance uses real API requests and persisted PostgreSQL data; locally authored test fixtures are only for development.